Practical no 1

Aim: Document Indexing and Retrieval

● Implement an inverted index construction algorithm.

● Build a simple document retrieval system using the constructed index.

```
from collections import defaultdict
document1 = "The quick brown fox jumped over the lazy dog."
document2 = "The lazy dog slept in the sun."
tokens1 = document1.lower().split()
tokens2 = document2.lower().split()
inverted_index = defaultdict(list)
for term in set(tokens1 + tokens2):
    if term in tokens1:
        inverted_index[term].append("Document 1")
    if term in tokens2:
        inverted_index[term].append("Document 2")
for term, docs in inverted_index.items():
    print(term, "->", ", ".join(docs))
```

Practical no 2

Aim: Retrieval Models

● Implement the Boolean retrieval model and process queries.

```
docs = {
    1: "apple banana orange",
    2: "apple banana",
    3: "banana orange",
    4: "apple"
}
# Build inverted index
index = {}
for i, t in docs.items():
    for w in t.split():
        index.setdefault(w, set()).add(i)
# Queries
print("Documents containing 'apple' and 'banana':", list(index["apple"] & index["banana"]))
```

```
print("Documents containing 'apple' or 'orange':", list(index["apple"] | index["orange"]))
print("Documents not containing 'orange':", list(set(docs) - index["orange"]))
```

● Implement the vector space model with TF-IDF weighting and cosine similarity.

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
docs = [
    "This is the first document.",
    "This document is the second document.",
    "And this is the third one.",
    "Is this the first document?"
]
query = "This is a query document."
# Convert documents + query into TF-IDF vectors
vectorizer = TfidfVectorizer()
tfidf = vectorizer.fit_transform(docs + [query])

# Compute cosine similarity between query and documents
similarities = cosine_similarity(tfidf[-1], tfidf[:-1])
print("Cosine Similarities:")
for i, score in enumerate(similarities[0]):
    print(f"Document {i+1}: {score}")
```

Practical no 3

Aim: Spelling Correction in IR Systems

● Develop a spelling correction module using edit distance algorithms.

● Integrate the spelling correction module into an information retrieval system.

(this will not show edit distance)

```
import difflib
dictionary = ["information", "retrieval", "spelling", "correction", "system"]
query = input("Enter your query: ")
matches = difflib.get_close_matches(query, dictionary, n=5, cutoff=0.6)
if matches:
    print("Did you mean:")
    for word in matches:
```

```
        print(word)
else:
    print("No suggestions found.")
```

(showing edit distance)

```
import difflib
dictionary = ["information", "retrieval", "spelling", "correction", "system"]
query = input("Enter your query: ").lower()
matches = difflib.get_close_matches(query, dictionary, n=5, cutoff=0.6)
if matches:
    print("Did you mean:")
    for word in matches:
        # Approximate edit distance from similarity ratio
        ratio = difflib.SequenceMatcher(None, query, word).ratio()
        distance = round((1 - ratio) * max(len(query), len(word)))
        print(f"{word} (Edit Distance: {distance})")
else:
    print("No suggestions found.")
```

Practical no 4

Aim: Evaluation Metrics for IR Systems

● Calculate precision, recall, and F-measure for a given set of retrieval Results.

```
from sklearn.metrics import precision_score, recall_score, f1_score
# Documents
docs = ["doc1","doc2","doc3","doc4"]
# True relevant documents
rel = {"doc1","doc4"}
# Retrieved documents
r = {"doc1","doc2","doc3"}

# Convert to binary labels (1 = yes, 0 = no)
y_true = [1 if d in rel else 0 for d in docs]
y_pred = [1 if d in r else 0 for d in docs]
```

```
print("Precision:", precision_score(y_true, y_pred))
print("Recall:", recall_score(y_true, y_pred))
print("F1:", f1_score(y_true, y_pred))
```

● Use an evaluation toolkit to measure average precision and other evaluation metrics.

```
from sklearn.metrics import *
import matplotlib.pyplot as plt
import numpy as np

y_true = [1, 0, 1, 0, 1]
y_prob = [0.8, 0.6, 0.9, 0.5, 0.7]
y_pred = (np.array(y_prob) >= 0.5).astype(int)

print("Accuracy:", accuracy_score(y_true, y_pred))
print("Precision:", precision_score(y_true, y_pred))
print("Recall:", recall_score(y_true, y_pred))
print("F1:", f1_score(y_true, y_pred))
print("ROC-AUC:", roc_auc_score(y_true, y_prob))
print("Avg Precision:", average_precision_score(y_true, y_prob))

p, r, _ = precision_recall_curve(y_true, y_prob)
plt.plot(r, p)
plt.xlabel("Recall")
plt.ylabel("Precision")
plt.title("PR Curve")
plt.show()
```

Practical no 5

Aim: Text Categorization

● Implement a text classification algorithm (e.g., Naive Bayes or Support Vector Machines).

● Train the classifier on a labeled dataset and evaluate its performance.

```
from sklearn.datasets import fetch_20newsgroups
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
```

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.svm import LinearSVC
from sklearn.metrics import *

# Load dataset
data = fetch_20newsgroups(subset='all', remove=('headers','footers','quotes'))

# Split
X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.2, random_state=42
)

# TF-IDF + SVM in one line
model = make_pipeline(TfidfVectorizer(), LinearSVC())
model.fit(X_train, y_train)

# Predict & Evaluate
y_pred = model.predict(X_test)

print("Accuracy:", round(accuracy_score(y_test, y_pred), 2))
print(classification_report(y_test, y_pred, target_names=data.target_names))
```

Practical no 6

Aim: Clustering for Information Retrieval

● Implement a clustering algorithm (e.g., K-means or hierarchical clustering).

● Apply the clustering algorithm to a set of documents and evaluate the clustering results.

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
docs = [
    "This is the first document.",
    "This document is the second document.",
    "And this is the third one.",
    "Is this the first document?",
```

```
    "The last document is here."
]
X = TfidfVectorizer().fit_transform(docs)
kmeans = KMeans(n_clusters=2, n_init='auto').fit(X)
labels = kmeans.labels_

print("Cluster labels:", labels)
print("Silhouette Score:", silhouette_score(X, labels))
```

Practical no 7

Aim: Web Crawling and Indexing

● Develop a web crawler to fetch and index web pages.

● Handle challenges such as robots.txt, dynamic content, and crawling delays.

```
import requests, time
from bs4 import BeautifulSoup
from urllib.parse import urljoin, urlparse
def crawl(seed, max_pages=10):
    base = f"{urlparse(seed).scheme}://{urlparse(seed).hostname}"
    visited, queue = set(), [seed]
    while queue and len(visited) < max_pages:
        url = queue.pop(0)
        if url not in visited:
            try:
                r = requests.get(url)
                print("Crawled:", url)
                soup = BeautifulSoup(r.text, "html.parser")
                for a in soup.find_all("a", href=True):
                    link = urljoin(url, a["href"])
                    if link.startswith(base):
                        queue.append(link)
                visited.add(url)
                time.sleep(1)
            except:
                pass
```

```
crawl("https://example.com", 10)
```

Practical no 8

```
import numpy as np
def pagerank(G, d=0.85, it=100, eps=1e-8):
    n = G.shape[0]
    M = G / G.sum(axis=1, keepdims=True)  # normalize outgoing links
    M = np.nan_to_num(M).T              # handle divide-by-zero & transpose
    r = np.ones(n) / n                  # initial rank

    for _ in range(it):
        new_r = (1-d)/n + d * M @ r
        if np.linalg.norm(new_r - r, 1) < eps:
            break
        r = new_r
    return r

G = np.array([
    [0,1,1,1],
    [1,0,0,0],
    [1,1,0,1],
    [0,0,1,0]
])

r = pagerank(G)

for i, s in enumerate(r):
    print(f"Page {i+1}: {s:.4f}")
```

Practical no 9

```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
```

```
from sklearn.metrics import ndcg_score

X,y = make_classification(n_samples=1000,n_features=10,n_classes=2,random_state=42)
Xtr, Xte, ytr,yte = train_test_split(X,y,test_size=0.2,random_state=42)
sc = StandardScaler()
Xtr, Xte = sc.fit_transform(Xtr), sc.transform(Xte)
model = SVC(kernel='linear').fit(Xtr,ytr)
pred = model.decision_function(Xte)
print("NDCG Score:", ndcg_score(yte.reshape(1,-1), pred.reshape(1,-1)))
```